Lab 19.3 – PEFT using LoRA

Install and Import Libraries

```
"""
STEP 1: Install Required Libraries
"""

!pip install torch transformers datasets peft accelerate -q

import torch

from transformers import (
    AutoModelForSequenceClassification,
    AutoTokenizer,
    TrainingArguments,
    Trainer
)

from datasets import load_dataset

from peft import LoraConfig, get_peft_model
```

Load Pretrained Model & Tokenizer

```
"""
STEP 2: Load Pre-trained Model and Tokenizer
"""

model_name = "distilbert-base-uncased"

tokenizer = AutoTokenizer.from_pretrained(model_name)

model = AutoModelForSequenceClassification.from_pretrained(
    model_name,
    num_labels=2
)

print("Model loaded successfully")
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: Use
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access pu
  warnings.warn(
Warning: You are sending unauthenticated requests to the HF Hub. Please set a
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated r
config.json: 100%                                          483/483 [00:00<00:00, 5.38kB/s]

tokenizer_config.json: 100%                                 48.0/48.0 [00:00<00:00, 695B/s]

vocab.txt:        232k/? [00:00<00:00, 1.73MB/s]

tokenizer.json:        466k/? [00:00<00:00, 4.14MB/s]

model.safetensors: 100%                                  268M/268M [00:04<00:00, 98.8MB/s]

Loading weights: 100%  100/100 [00:00<00:00, 236.07it/s, Materializing param=distilbert.transform

DistilBertForSequenceClassification LOAD REPORT from: distilbert-base-uncased
Key                      | Status     |
-------------------------+------------+-
vocab_layer_norm.bias    | UNEXPECTED |
vocab_projector.bias     | UNEXPECTED |
vocab_transform.bias     | UNEXPECTED |
vocab_layer_norm.weight  | UNEXPECTED |
vocab_transform.weight   | UNEXPECTED |
classifier.weight        | MISSING    |
classifier.bias          | MISSING    |
pre_classifier.weight    | MISSING    |
pre_classifier.bias      | MISSING    |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture;
- MISSING   :those params were newly initialized because missing from the chec
Model loaded successfully
```

Load and Preprocess Dataset

```
"""
STEP 3: Load and Preprocess Dataset
"""

dataset = load_dataset("imdb")

# Reduce dataset (important for lab)
train_dataset = dataset["train"].shuffle(seed=42).select(range(500))
test_dataset = dataset["test"].shuffle(seed=42).select(range(200))

def tokenize_function(example):
    return tokenizer(
        example["text"],
        truncation=True,
        padding="max_length",
        max_length=128
    )
```

```
train_dataset = train_dataset.map(tokenize_function, batched=True)
test_dataset = test_dataset.map(tokenize_function, batched=True)

# Format for PyTorch
train_dataset = train_dataset.remove_columns(["text"])
test_dataset = test_dataset.remove_columns(["text"])

train_dataset.set_format("torch")
test_dataset.set_format("torch")

print("Dataset ready")
```

```
README.md:          7.81k/?  [00:00<00:00,  217kB/s]
plain_text/train-00000-of-                          21.0M/21.0M  [00:00<00:00,  105MB/s]
00001.parquet:  100%
plain_text/test-00000-of-                           20.5M/20.5M  [00:00<00:00,  102MB/s]
00001.parquet:  100%
plain_text/unsupervised-                            42.0M/42.0M  [00:00<00:00,  109MB/s]
00000-of-00001.p(…):  100%
Generating  train  split:  100%                 25000/25000  [00:00<00:00,  52194.26  examples/s]
Generating  test  split:  100%                  25000/25000  [00:00<00:00,  61030.67  examples/s]
Generating  unsupervised  split:  100%          50000/50000  [00:00<00:00,  73536.58  examples/s]
Map:  100%                                          500/500  [00:00<00:00,  623.48  examples/s]
Map:  100%                                          200/200  [00:00<00:00,  941.84  examples/s]
Dataset ready
```

Configure LoRA

```
"""
STEP 4: Configure LoRA Adapter
"""

lora_config = LoraConfig(
    r=8,
    lora_alpha=16,
    target_modules=["q_lin", "v_lin"],  # important for DistilBERT
    lora_dropout=0.1,
    bias="none",
    task_type="SEQ_CLS"
)

print("LoRA configured")
```

```
LoRA configured
```

Apply PEFT Mode

```
"""
STEP 5: Apply PEFT Model
"""

peft_model = get_peft_model(model, lora_config)

# Show trainable params
peft_model.print_trainable_parameters()
```

```
WARNING:torchao.kernel.intmm:Warning: Detected no triton, on systems without T
trainable params: 739,586 || all params: 67,694,596 || trainable%: 1.0925
```

Training Configuration

```
"""
STEP 6: Define Training Arguments
"""

training_args = TrainingArguments(
    output_dir="./results",
    learning_rate=2e-4,
    per_device_train_batch_size=8,
    num_train_epochs=1,
    logging_steps=10,
    save_strategy="no"
)
```

Train Model

```
"""
STEP 6: Train the Model
"""

trainer = Trainer(
    model=peft_model,
    args=training_args,
    train_dataset=train_dataset
)

trainer.train()
```

```
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:775: Use
  super().__init__(loader)
```

[63/63 04:11, Epoch 1/1]

Step	Training Loss
10	0.718883
20	0.660224
30	0.670581
40	0.645663
50	0.658329
60	0.632141

```
TrainOutput(global_step=63, training_loss=0.6626754734251235, metrics=
{'train_runtime': 261.6387, 'train_samples_per_second': 1.911,
'train_steps_per_second': 0.241, 'total_flos': 16842425856000.0,
'train_loss': 0.6626754734251235, 'epoch': 1.0})
```

Evaluate Model

```
results = trainer.evaluate(eval_dataset=test_dataset)

print("Evaluation Results:")
print(results)
```

```
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:775: Us
  super().__init__(loader)
```

[25/25 00:46]

```
Evaluation Results:
{'eval_loss': 0.6459086537361145, 'eval_runtime': 51.6876, 'eval_samples_per_s
```

Prediction

```
"""
STEP 7: Test with Sample Input
"""

text = "The movie was absolutely amazing and inspiring!"

inputs = tokenizer(text, return_tensors="pt")

outputs = peft_model(**inputs)

prediction = torch.argmax(outputs.logits, dim=1)

print("Predicted Label:", prediction.item())
```

```
Predicted Label: 1
```